The Formal Grammar of Foliage: L-Systems for Procedural Generation of Computer Graphics

IS71021E Assignment 1

Oliver J. Martin
MSc Computer Games Programming

Department of Computing
Goldsmiths, University of London
November 27, 2025

1 Introduction

Despite originating as a mathematical theory for the development of filamentous organisms [1], the introduction of Lindenmayer systems (or L-systems) inspired a range of geometric interpretations with the aim of procedurally modelling complex branching structures such as plants and fractals [2]. In the ensuing decades, L-systems have been used in the modelling an ever-expanding range of structures - from city maps [3] to virtual creatures [4].
This paper describes an implementation of an L-system using Unity and the C# programming language. This system is capable of generating models in both 2D and 3D, as well as the incorporation of a lineage-based parametric system as first described in [5].

2 Methods

2.1 Parallel Rewriting

L-Systems are a type of string-rewriting mechanism, whereby a complex string is derived by successively replacing components of a simple initial string - with a set of production rules defining the replacements. As opposed to the sequential appplication of rules in Chomsky's earlier work on string rewriting [6], L-systems utilise *parallel rewriting* - where production rules are applied simultaneously to the entirety of the input string [1], opening up the possibility of generating more complex structures.

2.2 Turtle Geometry

To generate an image from the string generated by the L-system, a relative cursor known as a *turtle* is used to interpret each character of the final string in order - with different symbols representing different commands [2]. In the simplest case, the state of the turtle at each point is represented as a triplet (x, y, z, α) where (x, y, z) is the current position of the turtle and α represents the orientation of the turtle.

2.2.1 Orientation

In 2D, the orientation α can be represented by a single angle. However, to expand the capability of the system to generate 3D structures, the representaiton of the turtle's current orientation must be expanded to 3 orthonormal vectors $\vec{H}$,$\vec{L}$ and $\vec{U}$ - representing the heading of the turtle, the direction to the left, and the direction up respectively [7].

2.2.2 Rotation and Moving

Rotations of angle δ around each axis of the turtle's orientation are given by the following rotation matrices:

$$R_{\vec{H}}(\delta) = \begin{bmatrix} cos\,\delta & sin\,\delta & 0 \\ -sin\,\delta & cos\,\delta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$R_{\vec{L}}(\delta) = \begin{bmatrix} cos\,\delta & 0 & -sin\,\delta \\ 0 & 1 & 0 \\ sin\,\delta & 0 & cos\,\delta \end{bmatrix}$$

$$R_{\vec{U}}(\delta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & cos\,\delta & -sin\,\delta \\ 0 & sin\,\delta & cos\,\delta \end{bmatrix}$$

Representing the orientation axes as row vectors, the new orientation following a rotation can then be expressed as:

$$\begin{bmatrix} \vec{H}' \\ \vec{L}' \\ \vec{U}' \end{bmatrix} = R \begin{bmatrix} H_x & H_y & H_z \\ L_x & L_y & L_z \\ U_x & U_y & U_z \end{bmatrix} \tag{1}$$

The new position of of the turtle, (x', y', z') following a move forward of step size d can then be calculated as:

$$(x', y', z') = (x, y, z) + d\vec{H} \tag{2}$$

The following operations are then used to control the turtle:

DRAW	Move forward and draw a line
TURN	Rotate about $\vec{U}$, using $R_{\vec{U}}$. The only rotation used in 2D systems.
PITCH	Rotate about $\vec{L}$, using $R_{\vec{L}}$.
ROLL	Rotate about $\vec{H}$, using $R_{\vec{H}}$.
HORIZONTAL	Rotate turtle about its axis such that $\vec{U}$ is $(0, 1, 0)$.

2.2.3 Bracketed Systems

To achieve branching structures, a *bracketed* L-system is implemented. A *stack* is created and two new stack operations are introduced:

PUSH	Push the current turtle state onto the stack.
POP	Pop the top state from the stack and make it the current state of the turtle.

Typically, the push operation is represented with an opening square bracket ([ ) and the pop operation with a closing square bracket ( ]) - which together bracket and delimit branches within the string.

2.3 Edge and Node Rewriting

When using an L-system with turtle graphics, there are two common modes of operation - node and edge rewriting. Edge rewriting is simply replacing each edge (i.e. each symbol which represents a draw command) with a new sequence, while node rewriting involves replacing characters that are not graphically or physically relevant to the system (i.e. are ignored by the turtle) - with these characters instead being considered logically relevant and representing subfigures.

2.4 Parametric L-Systems

Parametric L-systems extend the capabilities of L-systems by associating numerical parameters with letters, which can control physical constants such as line length, line width, or rotation angle [**hanan**]. This allows for continous processes (such as a plant growing) to be modelled, as information can flow through the system as it "grows".

2.5 Implementation

The L-system was created primarily in C#, with the Unity game engine used to draw lines, create UI, and handle user inputs.

2.5.1 Objectives

- Read a configuration file containing L-system parameters, and display the result.
- Allow user input to control selected L-system, current iteration, and L-system parameters.
- Allow full customisation of symbols and commands within the configuration file.
- Extend L-system functionality into 3D.
- Implement parametric L-system functionality.

2.5.2 Classes

`LSystem`

Loads configuration files, runs the text replacement algorithm, and handles user input. For each loaded config file, the `RunSystem` function is run. For each iteration, this function instantiates a `Turtle` object, calls the `RunIteration` function, and gets that iteration's `Turtle` to draw the resulting string.

The `RunIteration` function takes the an initial string and a dictionary containing the production rules as inputs and computes the result of applying the rules. To do this, an empty "output" string is initialised and each character in the string is looped through. Each character is checked against the keys of the rules dictionary - if it exists as a key the associated value is appended to the output string (i.e. replacing that character with its rules), otherwise the original character is appended.

To propagate the parameters through the iterations, each character through the string actually checks the preceding character against the rules. In the case that an open bracket is the current character, the preceding character is stored until a closing bracket is encountered - with the current "parameter" values in between being added to a separate empty string in the meantime. Once the closing bracket is found, the relevant parameter values are found within the rule for the preceding character by performing a Regex match for any characters found between brackets. These values are then replaced with a custom match evaluator that splits the attributes by comma, multiplies the current attribute and the value in the rule together, and then reforms the correct format with the values seperated by commas. This updated rule is then added to the output string, which ensures the updated parameters are passed to the next iteration.

Once this process has been completed for the entire original string, the output string is returned.

`Turtle`

Handles the drawing of one iteration of a given L-system, given its string. Each turtle takes one string and a dictionary containing the commands associated with different characters as inputs, and starts by instantiating an initial `Line` object. Sequentially, each character in the string is checked against the commands dictionary. If a command is found, a switch statement checks the command's `type` property against the `CommandType.TYPES` enum to determine what type of operation to perform.

Similiarly to the `RunIteration` function, drawing is paused if an open bracket is encountered until the next closed bracket. Again, this reads the parameters found between the brackets and either multiplies the `amount` property of the command by the parameter (in the case of a length or angle parameter), or changes the width of the line at the current point (in the case of a width parameter).

For each character, the current position is stored as a `TurtlePoint` data structure - which can be added and retrieved from a stack of `TurtlePoints` if bracket commands are encountered. When a pop command is encountered, a new `Line` object is created as the new active line and is intialised with the popped `TurtlePoint` state.

The `HandleForward` method calculates the updated position of the turtle by utilising equation 2, and also updates the width of the line if it has been changed by a parameter - both of which are passed to the currently active `Line`.

The `HandleRotation` method constructs the correct rotation matrix from section 2.2.2, depending on the `type` property of the current command, this is then passed to the `Rotate` function of the current `TurtleOrientation` to calculate the new orientation of the turtle. The `HandleHorizontal` function rotates the turtle about its axis so that t $\vec{U}$ is $(0, 1, 0)$. To do this it uses the following equations:

$$\vec{L} = \frac{\vec{V} \times \vec{H}}{|\vec{V} \times \vec{H}|}$$

$$\vec{U} = \vec{H} \times \vec{L}$$

where $\vec{V}$ is the global up direction.

`Line`

The `Line` class primarily functions as an instantiable wrapper for the `LineRenderer` built-in class. It also handles processing the line width. When a point is added with a custom width, it is added to a `widths` dictionary - with its key being the cumulative length to that point along the line. When the line is completed, the `widthCurve` attribute of the `LineRenderer` is calculated - with the x value of each point being the proportional length of each point (i.e. the total length to that point divided by the length of the whole line), and the y value being the desired width. This ensures that each branch reduces in width at the correct rate.

2.5.3 Data Structures

`Config`

Contains the configuration details of an L-system.

Properties:

`name`	A string representing the name of the L-system.
`variables`	A list of strings containing the variables of the L-system.
`constants`	A list of strings containing the constants of the L-system.
`axiom`	A string representing the initial state of the L-system.
`rules`	A list of `ProductionRule` data structures containing the rules of the system.
`commands`	A list of `Command` data structures containing the operations associated with each symbol.
`n`	The integer number of iterations to compute.
`lineWidth`	The base line width, stored as a float.

Note: Symbols are represented as strings rather than characters for use in parametric systems, as well as potential future implementation of context-sensitivity. Additionally, Unity is not capable of serialising dictionaries, so both the `rules` and `commands` field act as wrappers which are later unwrapped into dictionaries for reduced time complexity.

The configuration is typically defined by a JSON configuration file. JSON was chosen as the format primarily due to its human-readability, as advanced system customisation through modifying configuration files was one of the objectives. The JSON file must contain all the fields detailed above.

`ProductionRule`

Contains a production rule for a given L-system.

Properties:

`key`	A string representing the character(s) to be replaced for a rule. If this is intended to be used as a module in a parametric system, the symbol should be followed by "()" (e.g. F()).
`value`	The value to replace the key with.

`Command`

Contains a single command for a given L-system.

Properties:

`key`	A string representing the indicator for the turtle to perform this command.
`type`	A `CommandType` value defining the type and amount of this command.

`CommandType`

Represents the type of a given command, and how much of that command to perform.

Properties:

`type`	A value from the `TYPES` enum to define the type of the command.
`amount`	A float defining the base amount of this command to perform (i.e. step size for a move command, angle for a rotation).

This class also contains the `TYPES` enum, which enumerates the different types of commands:

0	`DRAW`
1	`TURN`
2	`POP`
3	`PUSH`
4	`PITCH`
5	`ROLL`
6	`HORIZONTAL`

`TurtlePoint`

Represents the state of a turtle.

Properties:

`position`	The current position of the turtle represented as a Vector3.
`orientation`	A `TurtleOrientation` representing the current orientation.
`lineWidth`	The width of the line as a float.

`TurtleOrientation`

Contains the vectors $\vec{H}$, $\vec{L}$, and $\vec{U}$ for a turtle. Handles the calculation for rotations.

Properties:

`heading`	$\vec{H}$ represented as a Vector3.
`left`	$\vec{L}$ represented as a Vector3.
`up`	$\vec{U}$ represented as a Vector3.

Methods:

`Rotate`	Takes a 3x3 rotation matrix as an input, multiplies the rotation matrix by the current coordinate frame and returns the result as a a new `TurtleOrientation`

2.5.4 User Controls

Hot Keys

Right Arrow/D	Increment Iteration
Left Arrow/A	Decrement Iteration
Up Arrow/W	Next L-system
Down Arrow/S	Previous L-system
Scroll Wheel Up	Zoom Camera In
Scroll Wheel Down	Zoom Camera Out

GUI

Within the GUI, the user can modify any of the operations for the currently viewed L-system. This is done by selecting the desired parameter to change with the drop-down menu and then changing the value in the input field below. The system automatically redraws when the change is confirmed.

3 Results

For these results, the following symbols are used as commands:

F	Move Forwards and Draw
[	Push Current State to Stack
]	Pop State From Stack
+	Turn (Positive)
-	Turn (Negative)
&	Pitch (Positive)
^	Pitch(Negative)
\	Roll (Positive)
/	Roll (Negative)
—	Turn 180°
$	Set Up Vector to (0,1,0)

3.1 2D Plant-Like Structures

Figure 1 shows three plant-like structures generated by edge-rewriting L-systems, while figure 2 shows three-plant like structures generated by node-rewriting systems. Both figures use bracketed systems, which creates the branching structure that they demonstrate.

3.2 3D Hilbert Curve

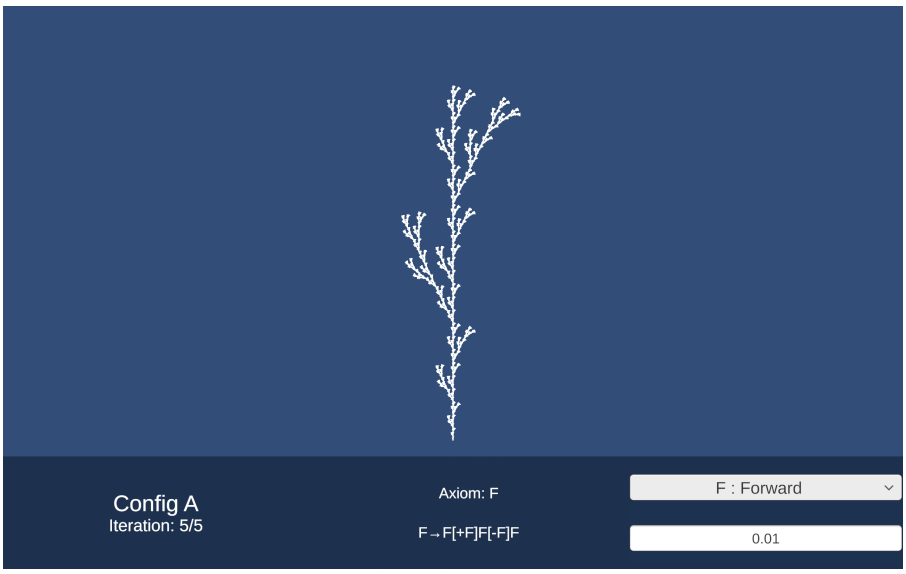
Figure 3 demonstrates the ability of this implementation to create complex 3D structures, with an extension of the Hilbert curve into 3D - as shown in [8]. This system implements 4 seperate node-replacement rules, with all rotations set to 90°. This structure is also not bracketed and consists of a single line - meaning it does not branch.

3.3 Monopodial Tree

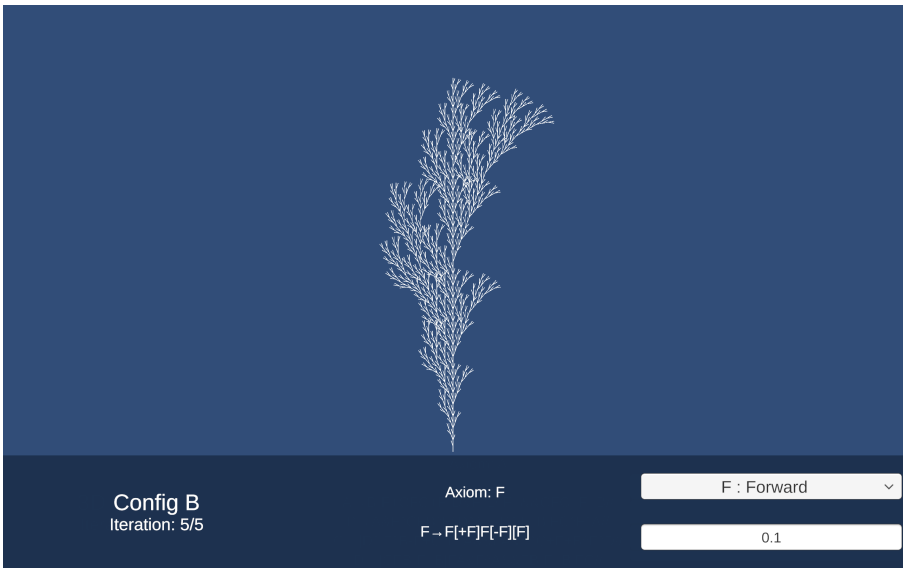
Figure 4 shows a model of a monopodial tree, generated by a bracketed, parametric, 3D L-system - again inspired by work from [8]. The following angles are used for the model shown in figure 4:

-	-45°
&	45°
/	137.5°

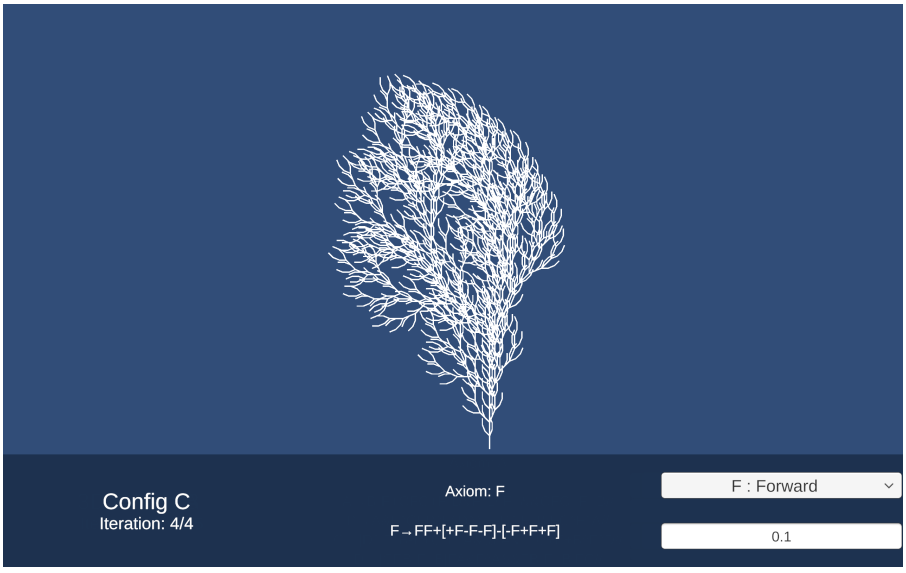
In this system − represents a branching angle from the lateral branches, & represents the branching angle from the trunk, and / represents the angle around the trunk between different branches. This creates a *monopodial* structure - where the main and lateral axes are clearly deliniated.
There are two parameters used in this system - one for line length and one for line width. The



(a) $\delta = 25.7°$

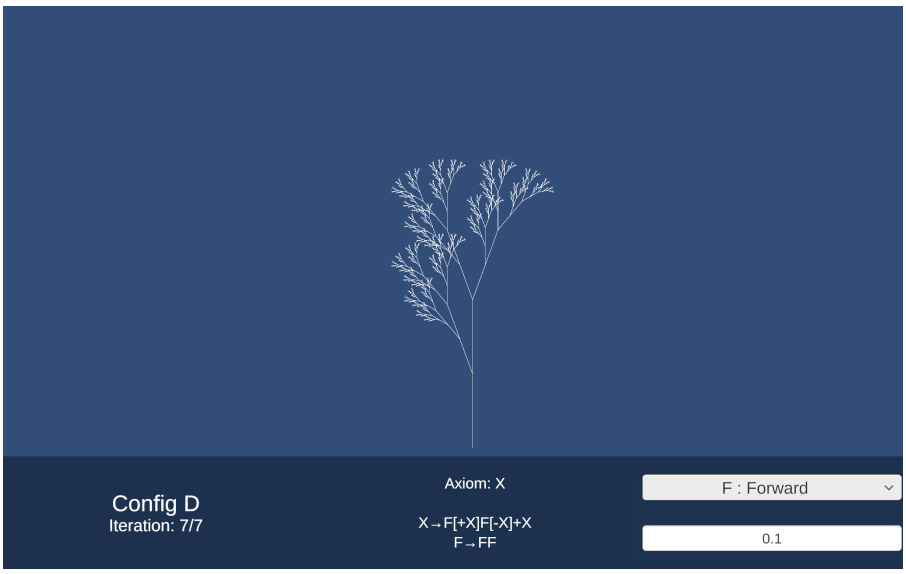


(b) $\delta = 20°$



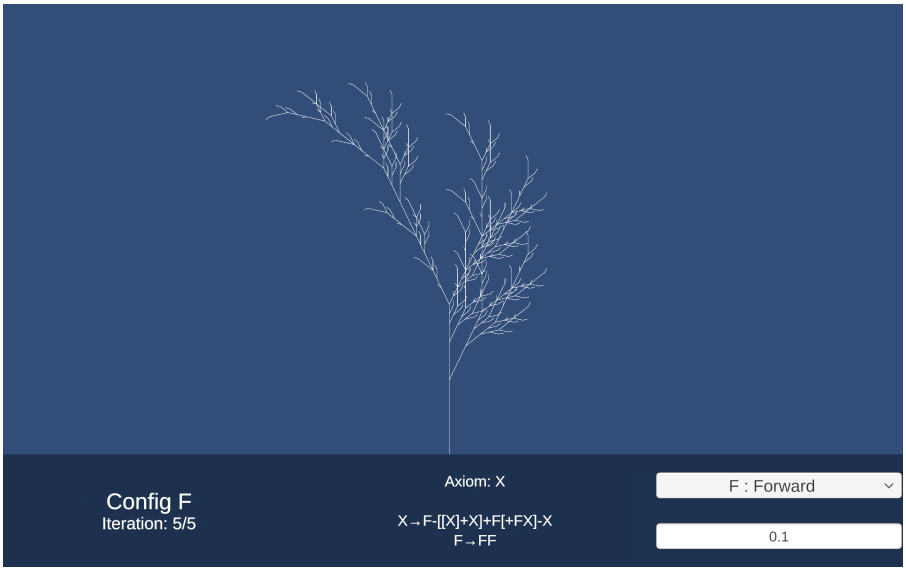
(c) $\delta = 22.5°$

Figure 1: Three 2D, Deterministic, Context-Free L-Systems Utilising Edge Replacement and Bracketing to Generate Branching Structures.



(a) $\delta = 20°$

(b) $\delta = 25.7°$



(c) $\delta = 22.5°$

Figure 2: Three 2D, Deterministic, Context-Free L-Systems Utilising Node Replacement and Bracketing to Generate Branching Structures.

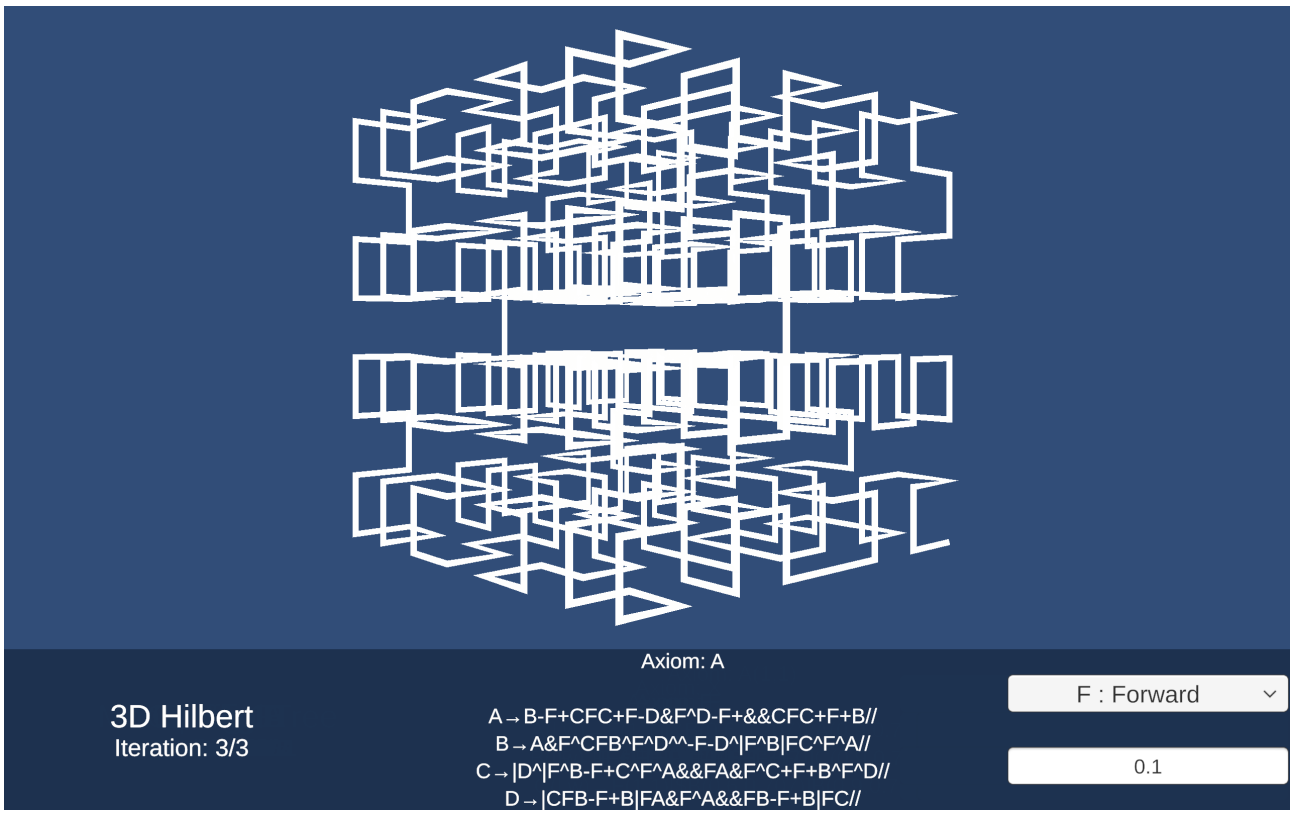


Figure 3: A 3D Extension of the Hilbert Curve, Inspired by [8].

trunk grows with a contraction ratio of 0.9 (meaning that new segments added to the trunk are 10% each iteration), while the branches have a contraction ratio of 0.6. The width, meanwhile, has a decreasing rate of 0.707 each iteration - ensuring that a "parent" branch has the same total diameter as two "child" branches [8]. This creates the effect seen in figure 4, where the branches continuosly get thinner as they get higher up the tree - something only possible due to the parametric aspect of the system.

4 Conclusion

The implemented system met all of the objectives detailed in section 2.5.1 - and is largely efficient, flexible, and robust. The monopodial tree in particular demonstrates the extent of the capablities of the system, as it includes all of the techniques discussed within this report.
There are a few possible routes for expansion of this implementation, the first of which would be the design of a system to generate sympodial trees such as those found in [8] - which would be as simple as creating a new configuration file. Additionally, expanding the GUI to be able to modify the axiom, rules, and parameters would allow even greater control over the produced model. Next, there is the possibility of introducing further expansions upon the basic L-system - such as stochasticism or context-sensitivity. All the models created in this report were deterministic, but adding a random element to the model generation would allow the system to be more useful in 3D modelling contexts. Context-sensitivity also opens the door to the creation of more complex models and, as mentioned, some of the capability is already in place in the current system to allow that. Finally, the camera controls within the program could use some work. Currently, the camera position is automatically set based on the average value of all the points of the model. This largely works, but for some particularly top-heavy models it can cause some to be cut off. The ability to zoom the camera was added to reduce this, but adding a fully controllable camera would improve the experience.

Figure 4: A Parametric L-System Used to Generate a Model of a Monopodial Tree.

References

[1] Aristid Lindenmayer. "Mathematical models for cellular interactions in development I. Filaments with one-sided inputs". In: *Journal of Theoretical Biology* 18.3 (1968), pp. 280–299. ISSN: 0022-5193. DOI: https://doi.org/10.1016/0022-5193(68)90079-9. URL: https://www.sciencedirect.com/science/article/pii/0022519368900799.

[2] P Prusinkiewicz. "Graphical applications of L-systems". In: *Proceedings on Graphics Interface '86/Vision Interface '86*. Vancouver, British Columbia, Canada: Canadian Information Processing Society, 1986, pp. 247–253.

[3] Yoav I. H. Parish and Pascal Müller. "Procedural modeling of cities". In: *Proceedings of the 28th Annual Conference on Computer Graphics and Interactive Techniques*. SIGGRAPH '01. New York, NY, USA: Association for Computing Machinery, 2001, pp. 301–308. ISBN: 158113374X. DOI: 10.1145/383259.383292. URL: https://doi.org/10.1145/383259.383292.

[4] Gregory S. Hornby and Jordan B. Pollack. "Evolving L-systems to generate virtual creatures". In: *Computers & Graphics* 25.6 (2001). Artificial Life, pp. 1041–1048. ISSN: 0097-8493. DOI: https://doi.org/10.1016/S0097-8493(01)00157-1. URL: https://www.sciencedirect.com/science/article/pii/S0097849301001571.

[5] James Scott Hanan. "Parametric l-systems and their application to the modelling and visualization of plants". AAINN83871. PhD thesis. 1992. ISBN: 031583871X.

[6] N. Chomsky. "Three models for the description of language". In: *IRE Transactions on Information Theory* 2.3 (1956), pp. 113–124. DOI: 10.1109/TIT.1956.1056813.

[7] Harold Abelson and Andrea diSessa. *Turtle Geometry: The Computer as a Medium for Exploring Mathematics*. The MIT Press, June 1981. ISBN: 9780262362740. DOI: 10.7551/mitpress/6933.001.0001. eprint: https://direct.mit.edu/book-pdf/2292124/book_9780262362740.pdf. URL: https://doi.org/10.7551/mitpress/6933.001.0001.

[8] P. Prusinkiewicz and Aristid Lindenmayer. *The algorithmic beauty of plants*. Berlin, Heidelberg: Springer-Verlag, 1990. ISBN: 0387972978.